Angular Reference

TERMS · ANALOGIES · TECHNICAL DEFINITIONS

Core Framework
RxJS / Async
HTTP / Routing
State (NgRx)
Meta / Runtime

CORE FRAMEWORK

TERM	ANALOGY	TECHNICAL DEFINITION
Angular	*The building's architecture blueprint*	Frontend framework by Google for building SPAs
Component	*A room in the building*	UI building block — class + template + styles
Template	*The room's interior design*	HTML with Angular syntax bound to the component class
Service	*Building plumbing — shared, runs behind walls*	Injectable singleton class for shared logic/data
Dependency Injection	*Calling reception instead of building it yourself*	Design pattern — Angular provides class instances via inject()
Standalone	*A room that brings its own furniture*	Component that declares its own imports, no NgModule needed
Module (legacy)	*A floor grouping related rooms*	NgModule — legacy grouping mechanism for components/services
Decorator	*A name badge clipped onto a class*	TypeScript metadata annotation — e.g. @Component, @Injectable
Input / Output	*Room door slot — data in, events out*	Component API — input() passes data down, output() emits events up
Lifecycle Hook	*Room alarm at key moments — entering, settling, leaving*	Interface methods Angular calls at defined points in a component's life

SIGNALS & CHANGE DETECTION

TERM	ANALOGY	TECHNICAL DEFINITION
Signal	*A live scoreboard — updates the exact screen watching*	Reactive primitive — tracked value that notifies dependants on change
Change Detection	*Building inspector walking every room*	Mechanism that syncs component state to the DOM

OnPush	*Inspector only visits if you call him*	CD strategy — skips component unless input ref changes or event fires
Zone.js	*Spy bug on every phone — reports when any call ends*	Third-party library Angular uses to intercept async APIs
Monkey Patch	*Secretly swapping the office phone with a bugged version*	Runtime override of a native browser API without caller knowing

RXJS / ASYNC

TERM	ANALOGY	TECHNICAL DEFINITION
Observable	*A news subscription — delivers whenever there's a story*	RxJS lazy data stream that emits values over time
Subscribe	*Actually opening your mailbox*	Method that starts execution of an Observable
Pipe	*Conveyor belt with filters before package reaches you*	RxJS .pipe() — chains operators to transform a stream
switchMap	*Cancelling your Uber and ordering a new one*	RxJS operator — cancels previous inner Observable, starts new one
tap	*Security camera — watches but doesn't touch*	RxJS operator — side effect only, passes value through unchanged
map	*Factory worker reshaping every item on the belt*	RxJS operator — transforms each emitted value
Async Pipe	*Butler who opens mail, reads it, cleans up after*	Angular pipe that subscribes, unwraps, and auto-unsubscribes

HTTP / ROUTING

TERM	ANALOGY	TECHNICAL DEFINITION
Router	*Building directory — tells you which room to go to*	Angular module managing client-side navigation via URL
Guard	*Bouncer at the room entrance*	Functional route guard — controls navigation access
Interceptor	*Customs — inspects every package in and out*	HTTP middleware — intercepts requests and responses globally

NGRX / STATE

TERM	ANALOGY	TECHNICAL DEFINITION
NgRx Store	*Single whiteboard everyone in the building reads from*	Redux-pattern global state container for Angular

Selector	*Highlighter on the whiteboard — shows only your section*	Pure function that derives a slice of state from the Store
Effect	*Task triggered when someone updates the whiteboard*	NgRx side effect handler — listens for actions, runs async logic

---

Angular Quick Reference · Modern Angular: Signals, inject(), Standalone

v17+ Signals No NgModules